CS7 - S03: CPU-GPU Workload Partitioning

BCSE412L Parallel Computing, Fall Semester 2026-27 | Individual Technical Report

Author: Aakaash Sathya Saravanan **Registration No.:** 23BCE0500 **Batch / Section:** CS7 **Date:** 19 October 2026

Code and data: https://github.com/ssaakaash/parallel-computing-cs7

Abstract

A GPU does the heavy lifting while the CPU sits idle and waits. Is that a waste, or does sharing the work cost more in copies and waiting than it saves? I measured how the share of tiles given to the CPU changes the makespan of an independent-tile workload, and where the best share sits. The workload is 256, 1024 and 4096 tiles of 256x256 floats, each blurred for 1 and 16 passes. A share of tiles from 0 to 100 percent runs on 1, 2 and 4 OpenMP threads and the rest on a Tesla T4 through CUDA. Medians of 7 repetitions are compared with CPU-serial, CPU-only and GPU-only baselines. At the largest size with 4 threads, the best split sent 70% and 20% of the tiles to the CPU (passes 1 and 16 respectively), a speedup of 2.54x and 1.25x over GPU-only. Over all 18 configurations the hybrid beat the faster single device by more than noise in 11. The optimal share fell by about 40 points as passes rose from 1 to 16, and the rule f* = T_gpu / (T_cpu + T_gpu) landed in the near-optimal range in 16 of 18 configurations.

1. Introduction and Literature Survey

Here is the problem. A discrete GPU runs a data-parallel kernel far faster than a multicore CPU, but the data has to cross the PCIe bus first, and the CPU sits idle the whole time. The CUDA Best Practices Guide says to minimise host-device transfers and overlap the remaining ones with computation [1]. Gregg and Hazelwood show that CPU-versus-GPU comparisons swing depending on where the data lives and whether transfer time is counted [4], and Mittal and Vetter name workload partitioning as one of the main ways to put both processors to work [2]. Qilin maps work onto the CPU and GPU using a model trained on measurements [3]. Nozal and Bosque extend oneAPI so that one kernel can co-execute on a CPU and an integrated GPU, and report that dynamic load balancing beats static splits [5]. Li et al. split each neural-network layer by output channels between CPU and GPU and use latency predictors to pick the split that balances the two devices [6]. Niaz et al. measure and model the energy of data transfer between CPU and GPU [7].

What is missing. Those works study runtimes, predictors or a single application. I wanted to isolate one question on a discrete PCIe GPU: for independent tiles, how does the best static CPU share move with arithmetic intensity, CPU thread count and problem size? And is a rule built from just two single-device timings good enough to find it? This picks up where the CPU-versus-GPU crossover study (S02) stops, by looking at everything between the two extremes.

2. Problem, Research Question and Hypotheses

Problem. N independent image tiles have to be processed with the smallest makespan, using the CPU, the GPU, or both. **Research question (CS7 S03).** Is CPU+GPU execution always better than GPU-only execution, and which split minimises the makespan? I went in with three hypotheses. **H1:** the best CPU share is neither 0 nor 100 percent, and it falls as arithmetic intensity (passes per tile) rises, because kernel work grows with passes while copy cost per tile does not. **H2:** the throughput-proportional share f* = T_gpu / (T_cpu + T_gpu), computed from the two single-device runs, predicts the measured optimum. **H3:** hybrid execution does not always beat GPU-only; where the CPU is slow next to the GPU, the gain disappears into noise.

3. Methodology

Workload. 256, 1024 and 4096 tiles of 256x256 single-precision pixels (64, 256 and 1024 MiB of input), each processed by 1 and 16 passes of a 3x3 box blur with clamped edges inside the tile. Passes set the arithmetic intensity while the data volume stays fixed. Tiles are independent, so any subset can run on either device. **Baselines (run before any splitting).** CPU-serial (1 thread, all tiles), CPU-only (all threads, f = 1) and GPU-only (f = 0). Speedup is measured against CPU-serial unless the text says GPU-only or CPU-only. **Partition.** A share f of the tiles goes to the CPU (OpenMP, 1, 2 and 4 threads, dynamic scheduling) and 1-f to the GPU (CUDA, 64-tile chunks: host-to-device copy, one kernel per pass, device-to-host copy, pinned host memory, 1 stream). The host queues the GPU work asynchronously, does its own share, then waits for the GPU. CPU shares from 0 to 100 percent were swept in 10-point steps. **Overheads.** Every run is split into computation (CPU compute time, GPU kernel time), communication and data movement (host-to-device plus device-to-host copies; on a PCIe GPU these are the same thing, and the CPU share has none because it works in shared memory), synchronisation (the host waiting for the GPU after its own share is done) and scheduling (partitioning plus queueing the GPU work). GPU times come from CUDA events. **Repeats and checks.** Each point was repeated 7 times with the sweep direction alternating. I report the median and show the spread as min-max and as a coefficient of variation. GPU output was checked against the CPU result for every share (maximum relative error 0.0e+00). **Metrics.** Speedup = baseline median / makespan; efficiency = speedup / (CPU threads that got work, plus 1 if the GPU got work); partition efficiency = ideal makespan 1/(1/T_cpu + 1/T_gpu) divided by measured makespan; CPU utilisation = busy thread time / (threads x makespan); GPU kernel utilisation = kernel time / makespan. **Environment.** GPU: Tesla T4, memory 15360 MiB, driver 580.178.04, target sm_75, GPUs visible: 2; CPU: Intel(R) Xeon(R) CPU @ 2.00GHz, logical CPUs 4; RAM 31.3 GB total, 30.1 GB available at start; nvcc 12.8 (V12.8.93); g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0; OS Linux-6.18.48+-x86_64-with-glibc2.39. Build: nvcc -O3 with host flags -fopenmp -O3 -march=native. Runtime: OMP_DYNAMIC=FALSE, CUDA_VISIBLE_DEVICES=0, no thread pinning.

Code and data. Source code, raw CSVs, plotting scripts, the environment record and the exact run commands are in the repository: https://github.com/ssaakaash/parallel-computing-cs7

4. Results and Discussion

Table 1. Summary at 4 CPU threads (medians, times in ms). Best share = CPU share with the lowest median makespan; range = shares within 2% of it. Speedups name their baseline in the column header. Eff. = efficiency of the best split (speedup over CPU serial / resources used).

Passes	Tiles	CPU-only	GPU-only	Best CPU share (range)	Best	vs GPU-only	vs CPU-only	vs CPU serial	Eff.	Predicted share f*
1	256	6.5	11.3	60% (60-70%)	4.5	2.49x	1.44x	3.44x	0.69	63%
1	1024	25.4	45.7	70%	18.2	2.51x	1.39x	3.58x	0.72	64%
1	4096	99.9	180.1	70%	71.0	2.54x	1.41x	3.33x	0.67	64%
16	256	78.3	20.0	20%	16.2	1.24x	4.84x	9.67x	1.93	20%
16	1024	293.0	80.3	20%	64.0	1.26x	4.58x	11.6x	2.33	22%
16	4096	1240.7	322.2	20%	258.0	1.25x	4.81x	11.1x	2.23	21%

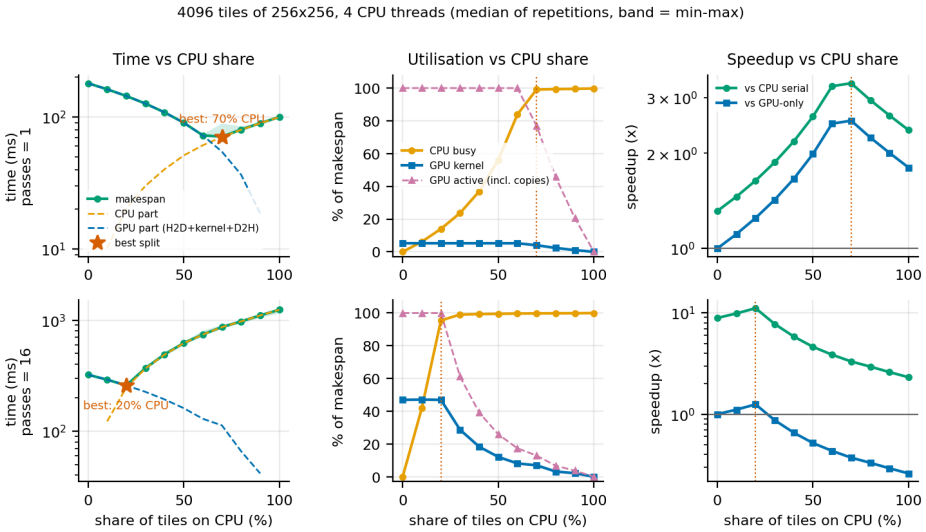


Figure 1. Makespan (with the CPU and GPU parts), utilisation and speedup against the share of tiles given to the CPU, at the largest size and thread count. Star = best share.

Table 2. Where the time goes at the best split (4096 tiles, 4 threads, medians, ms). The CPU and the GPU run at the same time, so the parts add up to more than the makespan.

Passes	CPU share	CPU compute	GPU kernels	Copies (H2D + D2H)	Host wait (sync)	Enqueue (sched.)	Makespan
1	70%	70.6	2.8	51.7	0.02	0.37	71.0
16	20%	247.0	121.2	136.6	7.83	2.94	258.0

The headline (4096 tiles, 4 threads). Passes 1: best share 70% CPU, 71.0 ms versus GPU-only 180 ms (2.54x) and CPU-only 99.9 ms (1.41x); Passes 16: best share 20% CPU, 258 ms versus GPU-only 322 ms (1.25x) and CPU-only 1241 ms (4.81x). Across all 18 configurations the best hybrid split beat GPU-only by more than the noise threshold (twice the larger coefficient of variation, at least 2%) in 11 and beat the faster of the two single devices in 11. The gain over GPU-only ranged from 1.11x to 2.54x, and the GPU was the faster single device in 12 of 18 configurations. **Utilisation at the best split:** passes 1: CPU busy 99%, GPU kernel 4%, GPU active including copies 77%; passes 16: CPU busy 95%, GPU kernel 47%, GPU active including copies 100%. **Scalability and efficiency:** adding CPU threads, the speedup of the best split over CPU serial grew from passes 1: 2.18x at 1 thread(s) to 3.33x at 4 (efficiency 0.67); passes 16: 9.95x at 1 thread(s) to 11.1x at 4 (efficiency 2.23). Efficiency here is speedup divided by (CPU threads + 1 GPU). It can go above 1 because the GPU does far more work than one CPU thread, so it should not be compared with a pure CPU efficiency.

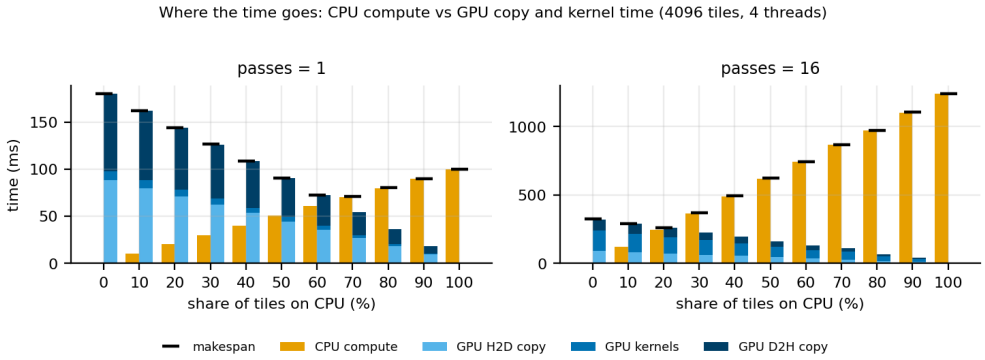


Figure 2. Time decomposition at each CPU share. Orange: CPU compute wall time. Blue stack: GPU host-to-device copy, kernels and device-to-host copy. Black dash: makespan.

Why it behaves this way. Look at where the GPU's time goes. In the GPU-only runs, copies took 95% of the GPU time at passes 1 and 53% at passes 16 (Figure 2). Copy cost per tile is fixed, but kernel cost grows with every extra pass, so the GPU gets cheaper per tile as the work per tile goes up. Over all configurations, the optimal CPU share fell by about 40 points from passes 1 to 16, rose by about 19 points from 1 to 4 threads, and stayed roughly constant from 256 to 4096 tiles. At interior optima the CPU and GPU parts finished within 8% of the makespan of each other (median), and the best split reached a median 95% of the ideal perfect-overlap makespan. The overheads I could separate were small at the best split (Table 2): the host's final wait for the GPU was 1.0% and the enqueue (scheduling) time 1.1% of the makespan (medians). The cost that matters is the copying, not the waiting.

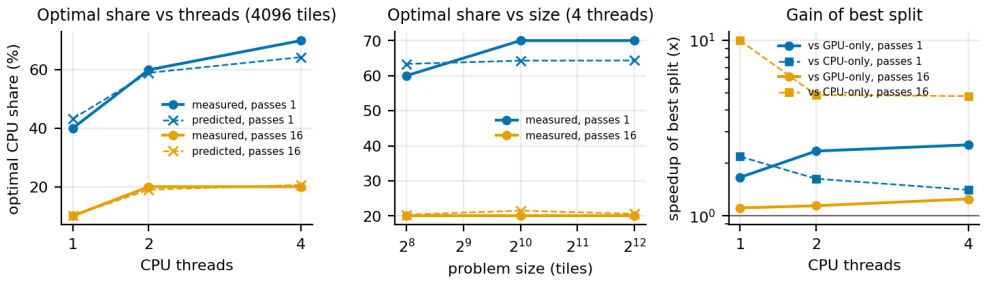


Figure 3. Optimal CPU share (solid, measured; dashed x, predicted f) against thread count and problem size, and the gain of the best split over each single device.*

How this lines up with the literature. The large copy share in the GPU-only runs (95% of GPU time at passes 1) is exactly what the CUDA guide [1] and Gregg and Hazelwood [4] warn about. Picking the split so that both devices finish together is the idea behind the trained model of Qilin [3] and the latency-predictor balancing of [6]. Here the simplest version of that idea, f*, matched the grid optimum closely (16 of 18 configurations). Nozal and Bosque [5] found dynamic balancing better than static splits. I only tested static splits, so I can neither confirm nor contradict that.

Verdict on the hypotheses. H1 is supported: the optimum was interior in 18 of 18 configurations and the optimal share fell by about 40 points from passes 1 to 16. H2 is supported: f* fell inside the near-optimal range (within 2% of the best makespan, plus or minus 5 points) in 16 of 18 configurations, with a mean absolute difference of 2 points from the grid optimum. H3 is supported: the hybrid did not beat GPU-only by more than noise in 7 of 18 configurations and did not beat the faster single device in 7. Run-to-run variation was a median 2.3% (maximum 16.3%) coefficient of variation of the makespan. The noisiest point (passes 16, 1024 tiles, 4 thread(s), 20% CPU share, median 64.0 ms, CV 16.3%) stayed in the data; I did not remove it.

5. Limitations and Threats to Validity

I had one GPU model, one CPU configuration and a shared cloud notebook, so the optimal shares belong to this hardware and should not be carried over to another machine (run-to-run variation went up to 16.3%). The notebook showed two GPUs; only the first was used. The workload is one regular, memory-light kernel, and irregular kernels would shift the balance. My CPU kernel is a hand-vectorised OpenMP loop and my GPU kernel is a simple global-memory blur, so the CPU/GPU throughput ratio is a property of these two implementations, not a law. Device buffers are allocated once and left out of the timing, and only the host-memory type and stream count listed in Section 3 were tested. Shares were swept on a 10-point grid, so optima are resolved to about 10 points, which is why I report the near-optimal range. OpenMP's own scheduling cost sits inside the CPU compute time and is not measured separately. Energy was not measured, so I can say nothing about the energy-optimal split. Thread counts above the physical core count share cores with the host thread that drives the GPU.

6. Conclusion and Future Work

For this tile-parallel workload CPU+GPU execution was not uniformly better: it beat the faster single device by more than noise in 11 of 18 configurations. At 4096 tiles and 4 threads the best split gave 2.54x and 1.25x over GPU-only (passes 1 and 16). The throughput-proportional rule is a usable starting point (16 of 18 configurations within the near-optimal range). Where I would go next: (1) replace the static split with dynamic chunk scheduling or work stealing and compare it with the best static split; (2) overlap copies and kernels with several CUDA streams and measure how far the optimum moves; (3) calibrate f* online from the first few tiles, as adaptive-mapping systems do [3, 6]; (4) add energy to the objective, since transfers and idle devices change the energy-optimal split [7].

References

[1] NVIDIA, "CUDA C++ Best Practices Guide," CUDA Toolkit Documentation, v13.0. https://docs.nvidia.com/cuda/archive/13.0.0/cuda-c-best-practices-guide/index.html

[2] S. Mittal and J. S. Vetter, "A Survey of CPU-GPU Heterogeneous Computing Techniques," ACM Computing Surveys, vol. 47, no. 4, Art. 69, 2015. https://doi.org/10.1145/2788396

[3] C.-K. Luk, S. Hong, and H. Kim, "Qilin: Exploiting Parallelism on Heterogeneous Multiprocessors with Adaptive Mapping," in Proc. 42nd IEEE/ACM Int. Symp. on Microarchitecture (MICRO-42), 2009, pp. 45-55.

[4] C. Gregg and K. Hazelwood, "Where is the Data? Why You Cannot Debate CPU vs. GPU Performance Without the Answer," in Proc. IEEE ISPASS, 2011. https://doi.org/10.1109/ISPASS.2011.5762730

[5] R. Nozal and J. L. Bosque, "Exploiting co-execution with oneAPI: heterogeneity from a modern perspective," arXiv:2106.01726, 2021 (Springer version: https://doi.org/10.1007/978-3-030-85665-6_31).

[6] Z. Li, M. Paolieri, and L. Golubchik, "Accelerating Mobile Inference through Fine-Grained CPU-GPU Co-Execution," in Selected Papers of EPEW 2025, LNCS vol. 15657, Springer, 2026, pp. 41-55.

[7] H. A. Niaz, R. R. Manumachu, and A. Lastovetsky, "Accurate and Reliable Energy Measurement and Modelling of Data Transfer Between CPU and GPU in Parallel Applications on Heterogeneous Hybrid Platforms," IEEE Trans. Computers, vol. 74, no. 3, pp. 1011-1024, 2025. https://doi.org/10.1109/TC.2024.3504262